

Znajdź kolejność

- **Limit czasu:** 10 minut
- **Środowisko:** 1 GPU (≈16 GB VRAM), bez dostępu do internetu
- **Rozmiar rozwiązania:** `solution.ipynb` ≤ 1 MB
- **Pamięć masowa:** 5 GB

Problem

Dane są mówione dialogi w języku angielskim między dwójgciem osób, *Mówcą A* i *Mówcą B*. Każdy dialog jest podzielony na wypowiedzi mówców (każda wypowiedź jest przypisana do konkretnego mówcy). Każda wypowiedź jest zapisana jako osobny plik audio `.wav`, zatem kompletny dialog jest reprezentowany przez zbiór plików `.wav`, po jednym dla każdej wypowiedzi.

Niestety wypowiedzi zostały losowo przetasowane, więc rozmowa nie ma już sensu. W nazwie pliku `chunk_{k}.wav`, k odnosi się do k -tego fragmentu w przetasowanym zbiorze, a nie do k -tej wypowiedzi w oryginalnym dialogu.

!! Twoim zadaniem jest odtworzenie oryginalnej chronologicznej kolejności rozmowy.



Zbiór danych

Każdy dialog zawiera n plików audio o nazwach `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`. Pliki to pojedyncze wypowiedzi. Nazwy plików odpowiadają wyłącznie przetasowanej kolejności. Nie wskazują one, gdzie dany fragment należy umieścić w oryginalnej rozmowie. Każdy dialog ma 7–20 fragmentów, mono, 44.1 kHz (możesz zmienić częstotliwość próbkowania).

`prefix.json` zawiera indeksy nazw plików pierwszych dwóch fragmentów w każdym dialogu. Pozwala to wskazać rzeczywisty początek dialogu i usuwa niejednoznaczność między odczytywaniem rozmowy w przód a odczytywaniem jej wstecz.

Na przykład: 11: [7, 12] oznacza, że pierwszą i drugą wypowiedzią dialogu 11 są odpowiednio `chunk_7.wav` oraz `chunk_12.wav`.

Co otrzymujesz

Otrzymujesz **dwa foldery w identycznym formacie**:

Folder	Dialogi	<code>answers.json</code> ?	Użyj go do
<code>dataset/train/</code>	1,288	✓ dołączony	trenowania / dostrajania (ang. fine-tuning) modelu
<code>dataset/test_public/</code>	100	✓ dołączony	uruchomiania pipeline'u i lokalnej oceny

Podczas oceniania folder `dataset/test_public/` jest w sposób niewidoczny zastępowany przez `hidden evaluation set` (`test_leaderboard_a` na potrzeby publicznej tabeli wyników oraz `test_leaderboard_b` na potrzeby końcowej tabeli wyników) — mają one taki sam rozmiar i format jak `dataset/test_public/`, ale bez `answers.json`.

Twój notebook jest ponownie wykonywany na tych danych, a utworzony przez niego plik `answers.json` jest używany do obliczenia wyniku. Wydzielone (ang. held-out) dialogi testowe pochodzą z tego samego rozkładu co `train`, więc lokalny wynik `test_public` stanowi wiarygodną zapowiedź wyniku.

Struktura katalogów

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} indeksy nazw plików
  answers.json # {dialogue_id: P} złota kolejność
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # obecne tylko na "development copy"
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

Wyjście

Dla każdego dialogu wyznacz oryginalną chronologiczną kolejność jego fragmentów audio. Twoja predykcja powinna być permutacją P zbioru $\{0, 1, \dots, n-1\}$, gdzie $P[i]$ jest przewidywaną pozycją chronologiczną `chunk_i.wav` (0 = pierwsza).

Twój plik wyjściowy `answers.json` powinien przypisywać każdemu identyfikatorowi dialogu (ang. dialogue ID) jego przewidywaną permutację:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

Przykład

Dialog ma 3 przetasowane fragmenty `chunk_0`, `chunk_1`, `chunk_2`:

przetasowany fragment	treść wypowiedzi	rzeczywista pozycja (ranga)
chunk_0.wav	"No worries — I'll send you the notes afterwards."	2 (ostatnia)
chunk_1.wav	"Hey, are you coming to the three o'clock meeting?"	0 (pierwsza)
chunk_2.wav	"I can't — I've got a dentist appointment then."	1

Rzeczywista kolejność to **chunk_1** → **chunk_2** → **chunk_0**, zatem $P = [2, 0, 1]$, a `prefix.json` zawiera $[1, 2]$.

⚠ P musi faktycznie być permutacją: o długości n , indeksowaną od 0, zawierającą każdą wartość dokładnie raz. Duplikaty, brakujące wartości lub wartości spoza zakresu (np. indeksowane od 1) skutkują wynikiem 0 za dany dialog, podobnie jak brak dialogu w pliku. Plik o nieprawidłowym formacie lub plik niebędący plikiem JSON zostaje odrzucony.

Ocenianie

Metryką oceny w tym zadaniu jest **trafność porządkowania parami** (ang. **pairwise ordering accuracy**). Sprawdza ona każdą parę fragmentów i zadaje pytanie: *który z nich powinien wystąpić pierwszy?* Para jest poprawna, jeśli twoja predykcja porządkuje ją tak samo jak złota kolejność (ang. ground truth). Dla dialogu zawierającego n fragmentów istnieje

$$M = n(n - 1)/2$$

par; niech I oznacza liczbę inwersji — par uporządkowanych inaczej niż w prawidłowym rozwiązaniu:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

i **Końcowy wynik jest średnią wyników dla poszczególnych dialogów w zbiorze.**

Dozwolone modele

Do rozwiązania tego zadania możesz używać wyłącznie następujących modeli pretrenowanych, zarówno podczas trenowania, jak i ewaluacji. Wszystkie te modele są już pobrane i dostępne w środowisku. Przykłady ich użycia znajdziesz w bazowym notebooku `solution.ipynb`. Pamiętaj, że nie możesz używać żadnego innego modelu, a twój program nie ma dostępu do internetu.

- **Reprezentacje mowy: wav2vec 2.0.** Enkoder modelu **Whisper** może być również używany jako ekstraktor cech (ang. feature extractor). [Karta modelu wav2vec](#)
- **Automatyczne rozpoznawanie mowy (ASR): OpenAI Whisper** (dowolny rozmiar). [Karta modelu Whisper](#)

- **Model językowy: Qwen2.5-0.5B**, którego można używać w trybie zero-shot albo dostroić na dostarczonym zbiorze `train`. [Karta modelu Qwen](#) Pamiętaj, że limit 10 minut musi obejmować zarówno trenowanie/dostrajanie które przeprowadzasz w czasie oceny rozwiązania, a także uruchomienie (inferencję) na zbiorze ewaluacyjnym.

Jak przesłać rozwiązanie

- Otwórz `solution.ipynb` i uruchom wszystkie komórki. Potwierdź, że zapisuje on `answers.json` w katalogu roboczym, z permutacją dla każdego dialogu w `dataset/test_public/` (100 dialogów). Podczas oceniania notebook zostaje ponownie uruchomiony na ukrytym zbiorze testowym, a utworzony przez niego plik `answers.json` zostanie tam oceniony.
- Ulepsz rozwiązanie jeśli chcesz—albo tego nie rób, bo sam baseline wystarcza do sprawdzenia, że pipeline działa.
- Otwórz kartę Git na lewym pasku bocznym JupyterLab.
- **Stage** `solution.ipynb` (ikona + obok niego).
- Wprowadź opis commita (ang. commit message) i kliknij **Commit**.
- Kliknij ikonę chmury ze strzałką w górę, aby zrobić push.
- Wróć na tę stronę konkursu i kliknij **Submit**.

Prześlij dokładnie jeden plik o nazwie `solution.ipynb`.